

## **Q1. University Database Normalization up to 3NF**

### **Aim**

To analyse a university database stored in a single table, identify normalization problems, and redesign the database from **Unnormalized Form (UNF)** to **Third Normal Form (3NF)** using the relational model.

### **1. Initial Table – Unnormalized Form (UNF)**

Suppose the university stores all student and course information in one table:

#### **STUDENT\_COURSE**

| Student_ID | Student_Name | Department | Course_ID | Course_Name | Faculty_Name | Faculty_Phone | Grade |
|------------|--------------|------------|-----------|-------------|--------------|---------------|-------|
| S101       | Aishwarya    | IT         | C101      | DBMS        | Dr. Kumar    | 9876543210    | A     |
| S101       | Aishwarya    | IT         | C102      | Python      | Dr. Ravi     | 9876543211    | B     |
| S102       | Priya        | CSE        | C101      | DBMS        | Dr. Kumar    | 9876543210    | A     |
| S103       | Rahul        | IT         | C103      | Networks    | Dr. Anil     | 9876543212    | B     |

### **2. Problems in the Table**

The single table contains information about **students, courses, and faculty** together.

#### **a) Data Redundancy**

Student and course information is repeated.

Example:

- DBMS and Dr. Kumar are repeated for every student taking C101.
- A student's name and department are repeated for every course.

#### **b) Update Anomaly**

If Dr. Kumar changes his phone number, we must update multiple rows.

If one row is not updated, inconsistent data will occur.

### c) Insertion Anomaly

A new course cannot be inserted easily if no student has enrolled in it yet.

### d) Deletion Anomaly

If the only student enrolled in a course is deleted, information about that course and faculty may also be lost.

### e) Functional Dependencies

The important dependencies are:

|                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student_ID $\rightarrow$ Student_Name, Department<br>Course_ID $\rightarrow$ Course_Name, Faculty_Name<br>Faculty_Name $\rightarrow$ Faculty_Phone<br>(Student_ID, Course_ID) $\rightarrow$ Grade |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The combination (Student\_ID, Course\_ID) identifies a student's enrollment and grade.

## **3. First Normal Form (1NF)**

### **Rule:**

A table is in **1NF** when:

- Every attribute contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The original table already contains atomic values, so we identify the composite key:

|                                       |
|---------------------------------------|
| Primary Key = (Student_ID, Course_ID) |
|---------------------------------------|

## 1NF Table

### STUDENT\_COURSE

| Student_ID | Student_Name | Department | Course_ID | Course_Name | Faculty_Name | Faculty_Phone | Grade |
|------------|--------------|------------|-----------|-------------|--------------|---------------|-------|
| S101       | Aishwarya    | IT         | C101      | DBMS        | Dr. Kumar    | 9876543210    | A     |
| S101       | Aishwarya    | IT         | C102      | Python      | Dr. Ravi     | 9876543211    | B     |
| S102       | Priya        | CSE        | C101      | DBMS        | Dr. Kumar    | 9876543210    | A     |
| S103       | Rahul        | IT         | C103      | Networks    | Dr. Anil     | 9876543212    | B     |

### Remaining problem

There are **partial dependencies**:

|                                        |
|----------------------------------------|
| Student_ID → Student_Name, Department  |
| Course_ID → Course_Name, Faculty_Name, |

These attributes depend only on part of the composite key.

### 4. Second Normal Form (2NF)

#### Rule:

A table must:

1. Be in 1NF.
2. Have no partial dependency on a composite key.

We divide the table into separate relations.

## STUDENT

| Student_ID | Student_Name | Department |
|------------|--------------|------------|
| S101       | Aishwarya    | IT         |
| S102       | Priya        | CSE        |
| S103       | Rahul        | IT         |

**Primary Key:** Student\_ID

## COURSE

| Course_ID | Course_Name | Faculty_Name | Faculty_Phone |
|-----------|-------------|--------------|---------------|
| C101      | DBMS        | Dr. Kumar    | 9876543210    |
| C102      | Python      | Dr. Ravi     | 9876543211    |
| C103      | Networks    | Dr. Anil     | 9876543212    |

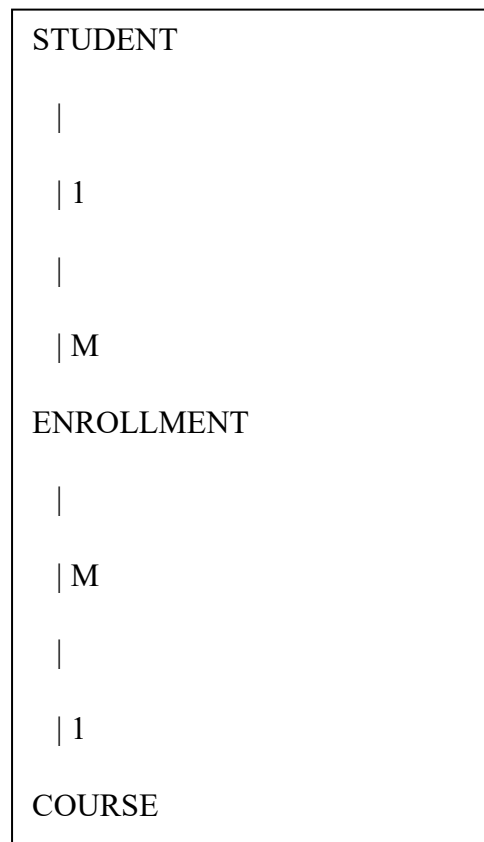
**Primary Key:** Course\_ID

## ENROLLMENT

| Student_ID | Course_ID | Grade |
|------------|-----------|-------|
| S101       | C101      | A     |
| S101       | C102      | B     |
| S102       | C101      | A     |
| S103       | C103      | B     |

**Composite Primary Key:** (Student\_ID, Course\_ID)

## Relationship



## **5. Third Normal Form (3NF)**

### **Rule:**

A table must:

1. Be in 2NF.
2. Have no transitive dependency.

In the COURSE table:

Course\_ID  $\rightarrow$  Faculty\_Name

Faculty\_Name  $\rightarrow$  Faculty\_Phone

Therefore:

Course\_ID  $\rightarrow$  Faculty\_Phone

This is a **transitive dependency**.

To remove it, create a separate FACULTY table.

### Final 3NF Relations

#### STUDENT

| Student_ID | Student_Name | Department |
|------------|--------------|------------|
| S101       | Aishwarya    | IT         |
| S102       | Priya        | CSE        |
| S103       | Rahul        | IT         |

**PK:** Student\_ID

#### FACULTY

| Faculty_ID | Faculty_Name | Faculty_Phone |
|------------|--------------|---------------|
| F01        | Dr. Kumar    | 9876543210    |
| F02        | Dr. Ravi     | 9876543211    |
| F03        | Dr. Anil     | 9876543212    |

**PK:** Faculty\_ID

## COURSE

| Course_ID | Course_Name | Faculty_ID |
|-----------|-------------|------------|
| C101      | DBMS        | F01        |
| C102      | Python      | F02        |
| C103      | Networks    | F03        |

**PK:**Course\_ID

**FK:** Faculty\_ID

## ENROLLMENT

| Student_ID | Course_ID | Grade |
|------------|-----------|-------|
| S101       | C101      | A     |
| S101       | C102      | B     |
| S102       | C101      | A     |
| S103       | C103      | B     |

**PK:**(Student\_ID,Course\_ID)

**FK:** Student\_ID, Course\_ID

## **6. Final Relational Schema**

STUDENT(  
Student\_ID PK,  
Student\_Name,  
Department  
)

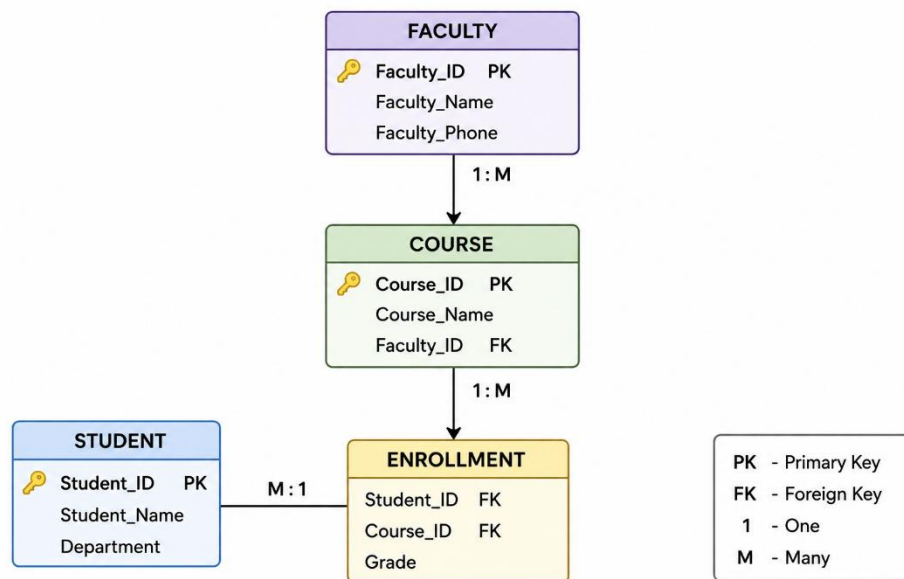
FACULTY(  
Faculty\_ID PK,  
Faculty\_Name,  
Faculty\_Phone  
)

COURSE(  
Course\_ID PK,  
Course\_Name,  
Faculty\_ID FK  
)

ENROLLMENT(  
Student\_ID PK/FK,  
Course\_ID PK/FK,  
Grade  
)



## Relationship Diagram



## FINAL SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 108
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE IF NOT EXISTS UniversityDB;
Query OK, 1 row affected, 1 warning (0.01 sec)

mysql> USE UniversityDB;
Database changed
mysql> DROP TABLE IF EXISTS Enrollment;
Query OK, 0 rows affected (0.06 sec)

mysql> DROP TABLE IF EXISTS Course;
Query OK, 0 rows affected (0.03 sec)

mysql> DROP TABLE IF EXISTS Faculty;
Query OK, 0 rows affected (0.03 sec)

mysql> DROP TABLE IF EXISTS Student;
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student (
->   Student_ID VARCHAR(10) PRIMARY KEY,
->   Student_Name VARCHAR(50) NOT NULL,
->   Department VARCHAR(30) NOT NULL
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Student (Student_ID, Student_Name, Department)
-> VALUES
-> ('S101', 'Aishwarya', 'IT'),
-> ('S102', 'Priya', 'CSE');
```

```
MySQL 8.0 Command Line Cli  x  +  v
-> ('S103', 'Rahul', 'IT'),
-> ('S104', 'Arun', 'ECE');
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+
| Student_ID | Student_Name | Department |
+-----+-----+-----+
| S101       | Aishwarya    | IT         |
| S102       | Priya        | CSE        |
| S103       | Rahul        | IT         |
| S104       | Arun         | ECE        |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE TABLE Faculty (
-> Faculty_ID VARCHAR(10) PRIMARY KEY,
-> Faculty_Name VARCHAR(50) NOT NULL,
-> Faculty_Phone VARCHAR(15)
-> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO Faculty (Faculty_ID, Faculty_Name, Faculty_Phone)
-> VALUES
-> ('F01', 'Dr. Kumar', '9876543210'),
-> ('F02', 'Dr. Ravi', '9876543211'),
-> ('F03', 'Dr. Anil', '9876543212');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Faculty;
+-----+-----+-----+
| Faculty_ID | Faculty_Name | Faculty_Phone |
+-----+-----+-----+
| F01        | Dr. Kumar    | 9876543210    |
| F02        | Dr. Ravi     | 9876543211    |
| F03        | Dr. Anil     | 9876543212    |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> CREATE TABLE Course (
-> Course_ID VARCHAR(10) PRIMARY KEY,
-> Course_Name VARCHAR(50) NOT NULL,
-> Faculty_ID VARCHAR(10) NOT NULL,
->
-> FOREIGN KEY (Faculty_ID)
-> REFERENCES Faculty(Faculty_ID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Course (Course_ID, Course_Name, Faculty_ID)
-> VALUES
-> ('C101', 'DBMS', 'F01'),
-> ('C102', 'Python', 'F02'),
-> ('C103', 'Computer Networks', 'F03');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Course;
+-----+-----+-----+
| Course_ID | Course_Name | Faculty_ID |
+-----+-----+-----+
| C101      | DBMS        | F01        |
| C102      | Python      | F02        |
| C103      | Computer Networks | F03        |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> CREATE TABLE Enrollment (
-> Student_ID VARCHAR(10),
-> Course_ID VARCHAR(10),
-> Grade CHAR(1),
->
-> PRIMARY KEY (Student_ID, Course_ID),
->
-> FOREIGN KEY (Student_ID)
-> REFERENCES Student(Student_ID),
->
-> FOREIGN KEY (Course_ID)
-> REFERENCES Course(Course_ID)
```

```
MySQL 8.0 Command Line Cli  x  +  v
-> REFERENCES Course(Course_ID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Enrollment (Student_ID, Course_ID, Grade)
-> VALUES
-> ('S101', 'C101', 'A'),
-> ('S101', 'C102', 'B'),
-> ('S102', 'C101', 'A'),
-> ('S103', 'C103', 'B'),
-> ('S104', 'C102', 'A');
Query OK, 5 rows affected (0.05 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| Student_ID | Course_ID | Grade |
+-----+-----+-----+
| S101       | C101      | A     |
| S101       | C102      | B     |
| S102       | C101      | A     |
| S103       | C103      | B     |
| S104       | C102      | A     |
+-----+-----+-----+
5 rows in set (0.04 sec)

mysql> SELECT
-> s.Student_ID,
-> s.Student_Name,
-> s.Department,
-> c.Course_ID,
-> c.Course_Name,
-> f.Faculty_Name,
-> e.Grade
-> FROM Student s
-> JOIN Enrollment e
-> ON s.Student_ID = e.Student_ID
-> JOIN Course c
-> ON e.Course_ID = c.Course_ID
-> JOIN Faculty f
-> ON c.Faculty_ID = f.Faculty_ID;
```

```
MySQL 8.0 Command Line Cli  x  +  v
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Course_ID | Course_Name | Faculty_Name | Grade |
+-----+-----+-----+-----+-----+-----+
| S101       | Aishwarya    | IT         | C101      | DBMS        | Dr. Kumar    | A     |
| S102       | Priya        | CSE        | C101      | DBMS        | Dr. Kumar    | A     |
| S101       | Aishwarya    | IT         | C102      | Python      | Dr. Ravi     | B     |
| S104       | Arun         | ECE        | C102      | Python      | Dr. Ravi     | A     |
| S103       | Rahul        | IT         | C103      | Computer Networks | Dr. Anil    | B     |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql> SELECT Student_ID, Student_Name
-> FROM Student
-> WHERE Department = 'IT';
+-----+-----+
| Student_ID | Student_Name |
+-----+-----+
| S101       | Aishwarya    |
| S103       | Rahul        |
+-----+-----+
2 rows in set (0.04 sec)

mysql> SELECT
-> s.Student_Name,
-> c.Course_Name,
-> e.Grade
-> FROM Student s
-> JOIN Enrollment e
-> ON s.Student_ID = e.Student_ID
-> JOIN Course c
-> ON e.Course_ID = c.Course_ID
-> WHERE e.Grade = 'A';
+-----+-----+-----+
| Student_Name | Course_Name | Grade |
+-----+-----+-----+
| Aishwarya    | DBMS        | A     |
| Priya        | DBMS        | A     |
| Arun         | Python      | A     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```

MySQL 8.0 Command Line Cli x + v
3 rows in set (0.00 sec)

mysql> SELECT
-> c.Course_ID,
-> c.Course_Name
-> FROM Course c
-> JOIN Faculty f
-> ON c.Faculty_ID = f.Faculty_ID
-> WHERE f.Faculty_Name = 'Dr. Kumar';
+-----+-----+
| Course_ID | Course_Name |
+-----+-----+
| C101      | DBMS        |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
-> c.Course_ID,
-> c.Course_Name,
-> COUNT(e.Student_ID) AS Number_of_Students
-> FROM Course c
-> LEFT JOIN Enrollment e
-> ON c.Course_ID = e.Course_ID
-> GROUP BY c.Course_ID, c.Course_Name;
+-----+-----+-----+
| Course_ID | Course_Name | Number_of_Students |
+-----+-----+-----+
| C101      | DBMS        | 2                  |
| C102      | Python      | 2                  |
| C103      | Computer Networks | 1                  |
+-----+-----+-----+
3 rows in set (0.05 sec)

mysql> UPDATE Faculty
-> SET Faculty_Phone = '9999999999'
-> WHERE Faculty_ID = 'F01';
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM Faculty;

```

```

MySQL 8.0 Command Line Cli x + v
+-----+-----+-----+
| F01      | Dr. Kumar   | 9999999999        |
| F02      | Dr. Ravi    | 9876543211        |
| F03      | Dr. Anil    | 9876543212        |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> DELETE FROM Enrollment
-> WHERE Student_ID = 'S104';
Query OK, 1 row affected (0.05 sec)

mysql> DELETE FROM Student
-> WHERE Student_ID = 'S104';
Query OK, 1 row affected (0.01 sec)

mysql> CREATE VIEW StudentCourseView AS
-> SELECT
-> s.Student_ID,
-> s.Student_Name,
-> s.Department,
-> c.Course_ID,
-> c.Course_Name,
-> f.Faculty_Name,
-> e.Grade
-> FROM Student s
-> JOIN Enrollment e
-> ON s.Student_ID = e.Student_ID
-> JOIN Course c
-> ON e.Course_ID = c.Course_ID
-> JOIN Faculty f
-> ON c.Faculty_ID = f.Faculty_ID;
Query OK, 0 rows affected (0.07 sec)

mysql> SELECT * FROM StudentCourseView;
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Course_ID | Course_Name | Faculty_Name | Grade |
+-----+-----+-----+-----+-----+-----+
| S101       | Aishwarya    | IT         | C101      | DBMS        | Dr. Kumar    | A     |
| S101       | Aishwarya    | IT         | C102      | Python      | Dr. Ravi     | B     |
| S102       | Priya        | CSE        | C101      | DBMS        | Dr. Kumar    | A     |
| S103       | Rahul        | IT         | C103      | Computer Networks | Dr. Anil     | B     |
+-----+-----+-----+-----+-----+-----+

```

## Result

The university's original single table was successfully redesigned into Student, Faculty, Course, and Enrollment tables in 3NF. The SQL implementation uses primary keys, foreign keys, joins, selection, projection, update, delete, and views, reducing redundancy and eliminating insertion, update, and deletion anomalies.

## **2)SQL Implementation: Employees Working in Multiple Departments**

### **1. Aim**

To design a company database using the relational model and write appropriate SQL queries to identify employees who work in multiple departments using JOIN, GROUP BY, COUNT, HAVING, subqueries, and views.

### **2. Relational Model**

The database contains three relations:

#### **EMPLOYEE**

|                                                         |
|---------------------------------------------------------|
| EMPLOYEE(Employee_ID, Employee_Name, Job_Title, Salary) |
|---------------------------------------------------------|

**Primary Key:** Employee\_ID

#### **DEPARTMENT**

|                                            |
|--------------------------------------------|
| DEPARTMENT(Department_ID, Department_Name) |
|--------------------------------------------|

**Primary Key:** Department\_ID

#### **EMPLOYEE\_DEPARTMENT**

|                                                 |
|-------------------------------------------------|
| EMPLOYEE_DEPARTMENT(Employee_ID, Department_ID) |
|-------------------------------------------------|

**Primary Key:** (Employee\_ID, Department\_ID)

**Foreign Keys:** Employee\_ID, Department\_ID

### **Relationship**

|                                                             |
|-------------------------------------------------------------|
| EMPLOYEE 1 ————— M EMPLOYEE_DEPARTMENT M ————— 1 DEPARTMENT |
|-------------------------------------------------------------|

This represents an M:N relationship between employees and departments.

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 109
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CompanyDB;
ERROR 1007 (HY000): Can't create database 'companydb'; database exists
mysql> USE CompanyDB;
Database changed
mysql> CREATE TABLE Employee (
->   Employee_ID INT PRIMARY KEY,
->   Employee_Name VARCHAR(50),
->   Job_Title VARCHAR(50),
->   Salary DECIMAL(10,2)
-> );
ERROR 1050 (42S01): Table 'employee' already exists
mysql> CREATE TABLE Department (
->   Department_ID INT PRIMARY KEY,
->   Department_Name VARCHAR(50)
-> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> CREATE TABLE Employee_Department (
->   Employee_ID INT,
->   Department_ID INT,
->   PRIMARY KEY (Employee_ID, Department_ID),
->   FOREIGN KEY (Employee_ID)
->     REFERENCES Employee(Employee_ID),
->   FOREIGN KEY (Department_ID)
->     REFERENCES Department(Department_ID)
-> );
```

```
MySQL 8.0 Command Line Cli  x  +  v
ERROR 1050 (42S01): Table 'employee_department' already exists
mysql> INSERT INTO Employee
-> (Employee_ID, Employee_Name, Job_Title, Salary)
-> VALUES
-> (101, 'Aishwarya', 'Software Engineer', 60000),
-> (102, 'Priya', 'Data Analyst', 55000),
-> (103, 'Rahul', 'Project Manager', 75000),
-> (104, 'Arun', 'Developer', 50000),
-> (105, 'Sneha', 'Business Analyst', 58000);
ERROR 1054 (42S22): Unknown column 'Employee_ID' in 'field list'
mysql> INSERT INTO Department
-> (Department_ID, Department_Name)
-> VALUES
-> (10, 'IT'),
-> (20, 'Finance'),
-> (30, 'HR'),
-> (40, 'Marketing');
ERROR 1054 (42S22): Unknown column 'Department_ID' in 'field list'
mysql> INSERT INTO Employee_Department
-> (Employee_ID, Department_ID)
-> VALUES
-> (101, 10),
-> (101, 30),
-> (102, 10),
-> (103, 20),
-> (103, 40),
-> (104, 10),
-> (105, 30),
-> (105, 40);
ERROR 1054 (42S22): Unknown column 'Employee_ID' in 'field list'
mysql> SELECT
->   e.Employee_ID,
->   e.Employee_Name,
->   COUNT(ed.Department_ID) AS Department_Count
-> FROM Employee e
-> JOIN Employee_Department ed
->   ON e.Employee_ID = ed.Employee_ID
-> GROUP BY
->   e.Employee_ID,
->   e.Employee_Name
-> HAVING COUNT(ed.Department_ID) > 1;
```

```
MySQL 8.0 Command Line Cli  x  +  v
ERROR 1054 (42S22): Unknown column 'e.Employee_ID' in 'field list'
mysql> USE CompanyDB;
Database changed
mysql> DROP TABLE IF EXISTS Employee_Department;
Query OK, 0 rows affected (0.14 sec)

mysql> DROP TABLE IF EXISTS Employee;
Query OK, 0 rows affected (0.04 sec)

mysql> DROP TABLE IF EXISTS Department;
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE Employee (
->   Employee_ID INT PRIMARY KEY,
->   Employee_Name VARCHAR(50) NOT NULL,
->   Job_Title VARCHAR(50),
->   Salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.08 sec)

mysql> DESC Employee;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| Employee_ID | int | NO | PRI | NULL | |
| Employee_Name | varchar(50) | NO | | NULL | |
| Job_Title | varchar(50) | YES | | NULL | |
| Salary | decimal(10,2) | YES | | NULL | |
+-----+-----+-----+-----+-----+
4 rows in set (0.10 sec)

mysql> CREATE TABLE Department (
->   Department_ID INT PRIMARY KEY,
->   Department_Name VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.09 sec)
```

```
MySQL 8.0 Command Line Cli x + v
mysql> DESC Department;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Department_ID | int | NO | PRI | NULL |  |
| Department_Name | varchar(50) | NO |  | NULL |  |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> CREATE TABLE Employee_Department (
-> Employee_ID INT,
-> Department_ID INT,
-> PRIMARY KEY (Employee_ID, Department_ID),
-> FOREIGN KEY (Employee_ID)
-> REFERENCES Employee(Employee_ID),
-> FOREIGN KEY (Department_ID)
-> REFERENCES Department(Department_ID)
-> );
Query OK, 0 rows affected (0.09 sec)

mysql> DESC Employee_Department;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Employee_ID | int | NO | PRI | NULL |  |
| Department_ID | int | NO | PRI | NULL |  |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> INSERT INTO Employee
-> (Employee_ID, Employee_Name, Job_Title, Salary)
-> VALUES
-> (101, 'Aishwarya', 'Software Engineer', 60000),
-> (102, 'Priya', 'Data Analyst', 55000),
-> (103, 'Rahul', 'Project Manager', 75000),
-> (104, 'Arun', 'Developer', 50000),
-> (105, 'Sneha', 'Business Analyst', 58000);
Query OK, 5 rows affected (0.02 sec)

MySQL 8.0 Command Line Cli x + v
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| Employee_ID | Employee_Name | Job_Title | Salary |
+-----+-----+-----+-----+
| 101 | Aishwarya | Software Engineer | 60000.00 |
| 102 | Priya | Data Analyst | 55000.00 |
| 103 | Rahul | Project Manager | 75000.00 |
| 104 | Arun | Developer | 50000.00 |
| 105 | Sneha | Business Analyst | 58000.00 |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql> INSERT INTO Department
-> (Department_ID, Department_Name)
-> VALUES
-> (10, 'IT'),
-> (20, 'Finance'),
-> (30, 'HR'),
-> (40, 'Marketing');
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Department;
+-----+-----+
| Department_ID | Department_Name |
+-----+-----+
| 10 | IT |
| 20 | Finance |
| 30 | HR |
| 40 | Marketing |
+-----+-----+
4 rows in set (0.12 sec)

mysql> INSERT INTO Employee_Department
-> (Employee_ID, Department_ID)
-> VALUES
-> (101, 10),
-> (101, 30),
-> (102, 10),
-> (103, 20),
-> (103, 40),
```

```
MySQL 8.0 Command Line Cli
-> (104, 10),
-> (105, 30),
-> (105, 40);
Query OK, 8 rows affected (0.11 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Employee_Department;
+-----+-----+
| Employee_ID | Department_ID |
+-----+-----+
| 101 | 10 |
| 102 | 10 |
| 104 | 10 |
| 103 | 20 |
| 101 | 30 |
| 105 | 30 |
| 103 | 40 |
| 105 | 40 |
+-----+-----+
8 rows in set (0.00 sec)

mysql> SELECT
-> e.Employee_ID,
-> e.Employee_Name,
-> COUNT(ed.Department_ID) AS Department_Count
-> FROM Employee e
-> JOIN Employee_Department ed
-> ON e.Employee_ID = ed.Employee_ID
-> GROUP BY
-> e.Employee_ID,
-> e.Employee_Name
-> HAVING COUNT(ed.Department_ID) > 1;
+-----+-----+-----+
| Employee_ID | Employee_Name | Department_Count |
+-----+-----+-----+
| 101 | Aishwarya | 2 |
| 103 | Rahul | 2 |
| 105 | Sneha | 2 |
+-----+-----+-----+
3 rows in set (0.03 sec)

mysql> SELECT
-> e.Employee_ID,
-> e.Employee_Name,
-> GROUP_CONCAT(d.Department_Name
-> ORDER BY d.Department_Name
-> SEPARATOR ', ') AS Departments
-> FROM Employee e
-> JOIN Employee_Department ed
-> ON e.Employee_ID = ed.Employee_ID
-> JOIN Department d
-> ON ed.Department_ID = d.Department_ID
-> GROUP BY
-> e.Employee_ID,
-> e.Employee_Name
-> HAVING COUNT(DISTINCT d.Department_ID) > 1;
+-----+-----+-----+
| Employee_ID | Employee_Name | Departments |
+-----+-----+-----+
| 101 | Aishwarya | HR, IT |
| 103 | Rahul | Finance, Marketing |
| 105 | Sneha | HR, Marketing |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

## Justification of the Approach

The Employee and Department tables have a many-to-many relationship because:

- One employee can work in multiple departments.
- One department can have multiple employees.

Therefore, an intermediate table called Employee\_Department is required.



The main SQL query uses:

| SQL Operation | Purpose                                              |
|---------------|------------------------------------------------------|
| JOIN          | Connects employees with their department assignments |
| GROUP BY      | Groups records employee-wise                         |
| COUNT()       | Counts the number of departments                     |
| HAVING        | Filters employees having more than one department    |
| DISTINCT      | Avoids duplicate department counting                 |
| VIEW          | Stores the query result as a virtual table           |

HAVING is used instead of WHERE because the condition is based on the aggregate value COUNT().

### **Conclusion**

The company database was designed using the relational model, and SQL queries using JOIN, GROUP BY, COUNT, HAVING, and subqueries were applied to retrieve employees working in multiple departments. A view was also created to simplify repeated access to the result. The approach is efficient, avoids redundancy, and correctly represents the many-to-many relationship between employees and departments.

### **3) Candidate Keys and BCNF Decomposition**

#### **1. Given Relation**

Consider the relation:

|                                                                               |
|-------------------------------------------------------------------------------|
| R(Student_ID, Course_ID, Student_Name, Course_Name, Faculty_ID, Faculty_Name) |
|-------------------------------------------------------------------------------|

#### **Functional Dependencies**

Assume the following functional dependencies:

|                                                     |
|-----------------------------------------------------|
| F1: Student_ID $\rightarrow$ Student_Name           |
| F2: Course_ID $\rightarrow$ Course_Name, Faculty_ID |
| F3: Faculty_ID $\rightarrow$ Faculty_Name           |

We need to:

1. Find the candidate key.
2. Check whether the relation is in BCNF.
3. If it is not in BCNF, decompose it.

## **2.Find Candidate Key**

The attributes of the relation are:

|              |
|--------------|
| Student_ID   |
| Course_ID    |
| Student_Name |
| Course_Name  |
| Faculty_ID   |
| Faculty_Name |

Start with:

|                         |
|-------------------------|
| {Student_ID, Course_ID} |
|-------------------------|

### **Step 1**

From:

|                                       |
|---------------------------------------|
| Student_ID $\rightarrow$ Student_Name |
|---------------------------------------|

we get:

|              |
|--------------|
| Student_Name |
|--------------|

So:

$$\{Student\_ID, Course\_ID\}^+ = \{Student\_ID, Course\_ID, Student\_Name\}$$

## Step 2

From:

$$Course\_ID \rightarrow Course\_Name, Faculty\_ID$$

we get:

$$\begin{matrix} Course\_Name \\ Faculty\_ID \end{matrix}$$

Now:

$$\begin{aligned} &\{Student\_ID, Course\_ID\}^+ \\ &= \\ &\{Student\_ID, Course\_ID, \\ &\quad Student\_Name, Course\_Name, Faculty\_ID\} \end{aligned}$$

## Step 3

From:

$$Faculty\_ID \rightarrow Faculty\_Name$$

we get:

$$Faculty\_Name$$

Therefore:

$$\{Student\_ID, Course\_ID\}^+ = \{Student\_ID, Course\_ID, Student\_Name, Course\_Name, Faculty\_ID, Faculty\_Name\}$$

This contains all attributes of R.

Therefore:

### **Candidate Key**

$$(Student\_ID, Course\_ID)$$

is a candidate key.

### **3. Check BCNF**

#### **BCNF Rule**

A relation is in BCNF if, for every non-trivial functional dependency:

$$X \rightarrow Y$$

X must be a superkey.

#### **FD 1**

$$Student\_ID \rightarrow Student\_Name$$

Is Student\_ID a superkey?

**No.**

Student\_ID alone cannot determine all attributes.

$$Student\_ID \rightarrow Student\_Name$$

## FD 2

Course\_ID  $\rightarrow$  Course\_Name, Faculty\_ID

Is Course\_ID a superkey?

No.

Therefore this dependency also violates BCNF.

## FD 3

Faculty\_ID  $\rightarrow$  Faculty\_Name

Is Faculty\_ID a superkey?

No.

Therefore this dependency also violates BCNF.

## Conclusion

R is NOT in BCNF.

## 4. BCNF Decomposition

We decompose the relation based on the violating dependencies.

### Decomposition 1 – Student

Using:

Student\_ID  $\rightarrow$  Student\_Name

Create:

STUDENT(Student\_ID, Student\_Name)

Here:

Student\_ID  $\rightarrow$  Student\_Name

and Student\_ID is the key of this relation.

So this relation is in BCNF.

### **Decomposition 2 – Faculty**

Using:

$\text{Faculty\_ID} \rightarrow \text{Faculty\_Name}$

Create:

$\text{FACULTY}(\text{Faculty\_ID}, \text{Faculty\_Name})$

and Faculty\_ID is the key.

Therefore, this relation is in BCNF.

### **Decomposition 3 – Course**

Using:

$\text{Course\_ID} \rightarrow \text{Course\_Name}, \text{Faculty\_ID}$

Create:

$\text{COURSE}(\text{Course\_ID}, \text{Course\_Name}, \text{Faculty\_ID})$

Here:

$\text{Course\_ID} \rightarrow \text{Course\_Name}, \text{Faculty\_ID}$

and Course\_ID is the key.

Therefore, this relation is also in BCNF.

### **Decomposition 4 – Enrollment**

The original candidate key was:

$(\text{Student\_ID}, \text{Course\_ID})$

Therefore, create:

ENROLLMENT(Student\_ID, Course\_ID)

This represents which students take which courses.

## SQL Implementation

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 118
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE BCNF_DB;
Query OK, 1 row affected (0.06 sec)

mysql> USE BCNF_DB;
Database changed
mysql> CREATE TABLE Student (
->   Student_ID INT PRIMARY KEY,
->   Student_Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.20 sec)

mysql> CREATE TABLE Faculty (
->   Faculty_ID INT PRIMARY KEY,
->   Faculty_Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE Course (
->   Course_ID INT PRIMARY KEY,
->   Course_Name VARCHAR(50),
->   Faculty_ID INT,
->   FOREIGN KEY (Faculty_ID)
->     REFERENCES Faculty(Faculty_ID)
-> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Enrollment (
->   Student_ID INT,
->   PRIMARY KEY (Student_ID, Course_ID),
->   FOREIGN KEY (Student_ID)
->     REFERENCES Student(Student_ID),
->   FOREIGN KEY (Course_ID)
->     REFERENCES Course(Course_ID)
-> );
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO Student VALUES
-> (101, 'Aishwarya'),
-> (102, 'Priya'),
-> (103, 'Rahul');
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Faculty VALUES
-> (1, 'Dr. Kumar'),
-> (2, 'Dr. Ravi');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Course VALUES
-> (201, 'DBMS', 1),
-> (202, 'Python', 2);
Query OK, 2 rows affected (0.04 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Enrollment VALUES
-> (101, 201),
-> (101, 202),
-> (102, 201),
-> (103, 202);
Query OK, 4 rows affected (0.08 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT
->   s.Student_ID,
->   s.Student_Name,
->   c.Course_ID,
->   c.Course_Name,
->   f.Faculty_Name
-> FROM Student s
-> JOIN Enrollment e
->   ON s.Student_ID = e.Student_ID
-> JOIN Course c
->   ON e.Course_ID = c.Course_ID
-> JOIN Faculty f
->   ON c.Faculty_ID = f.Faculty_ID;
+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Course_ID | Course_Name | Faculty_Name |
+-----+-----+-----+-----+-----+
| 101 | Aishwarya | 201 | DBMS | Dr. Kumar |
| 102 | Priya | 201 | DBMS | Dr. Kumar |
| 101 | Aishwarya | 202 | Python | Dr. Ravi |
| 103 | Rahul | 202 | Python | Dr. Ravi |
+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

#### **4)Banking System: Views and Constraints for Security and Integrity**

##### **1. Aim**

To design a secure banking database using SQL views and integrity constraints so that different types of users can access only the information they are authorized to view or modify while maintaining data accuracy and consistency.

##### **2. Scenario**

A banking system has three types of users:

1. **Customer** – can view their own account and transaction details.
2. **Bank Employee** – can view customer/account information required for banking operations.
3. **Bank Manager** – has broader access to account and transaction information.

The database contains:

|             |
|-------------|
| CUSTOMER    |
| ACCOUNT     |
| TRANSACTION |

##### **3. Relational Model**

###### **CUSTOMER**

|                                                                        |
|------------------------------------------------------------------------|
| CUSTOMER(<br>Customer_ID PK,<br>Customer_Name,<br>Email,<br>Phone<br>) |
|------------------------------------------------------------------------|



## ACCOUNT

```
ACCOUNT(  
    Account_No PK,  
    Customer_ID FK,  
    Account_Type,  
    Balance  
)
```

## TRANSACTION

```
TRANSACTION(  
    Transaction_ID PK,  
    Account_No FK,  
    Transaction_Type,  
    Amount,  
    Transaction_Date  
)
```

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 111
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE BankingDB;
Query OK, 1 row affected (0.08 sec)

mysql> USE BankingDB;
Database changed
mysql> DROP DATABASE BankingDB;
Query OK, 0 rows affected (0.13 sec)

mysql> CREATE DATABASE BankingDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE BankingDB;
Database changed
mysql> CREATE TABLE Customer (
->   Customer_ID INT PRIMARY KEY,
->   Customer_Name VARCHAR(50) NOT NULL,
->   Email VARCHAR(100) UNIQUE,
->   Phone VARCHAR(15) UNIQUE
-> );
Query OK, 0 rows affected (0.14 sec)

mysql> CREATE TABLE Account (
->   Account_No BIGINT PRIMARY KEY,
->   Customer_ID INT NOT NULL,
->   Account_Type VARCHAR(20) NOT NULL,
->   Balance DECIMAL(12,2) NOT NULL DEFAULT 0.00,
->
->   CONSTRAINT chk_account_type
```

```
MySQL 8.0 Command Line Cli  x  +  v
->   CHECK (Account_Type IN ('Savings', 'Current')),
->   CONSTRAINT chk_balance
->   CHECK (Balance >= 0),
->   FOREIGN KEY (Customer_ID)
->     REFERENCES Customer(Customer_ID)
-> );
Query OK, 0 rows affected (0.12 sec)

mysql> CREATE TABLE Bank_Transaction (
->   Transaction_ID INT PRIMARY KEY,
->   Account_No BIGINT NOT NULL,
->   Transaction_Type VARCHAR(20) NOT NULL,
->   Amount DECIMAL(12,2) NOT NULL,
->   Transaction_Date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->
->   CONSTRAINT chk_transaction_type
->     CHECK (Transaction_Type IN ('Deposit', 'Withdrawal')),
->   CONSTRAINT chk_amount
->     CHECK (Amount > 0),
->   FOREIGN KEY (Account_No)
->     REFERENCES Account(Account_No)
-> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Customer
-> (Customer_ID, Customer_Name, Email, Phone)
-> VALUES
-> (101, 'Aishwarya', 'aishwarya@gmail.com', '9876543210'),
-> (102, 'Priya', 'priya@gmail.com', '9876543211'),
-> (103, 'Rahul', 'rahul@gmail.com', '9876543212');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Account
-> (Account_No, Customer_ID, Account_Type, Balance)
-> VALUES
-> (10001, 101, 'Savings', 50000.00),
```

```

MySQL 8.0 Command Line Cli  x  +  v
-> (10003, 103, 'Current', 75000.00);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE VIEW Customer_Account_View AS
-> SELECT
-> c.Customer_ID,
-> c.Customer_Name,
-> a.Account_No,
-> a.Account_Type,
-> a.Balance
-> FROM Customer c
-> JOIN Account a
-> ON c.Customer_ID = a.Customer_ID;
Query OK, 0 rows affected (0.07 sec)

mysql> SELECT * FROM Customer_Account_View;
+-----+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Account_No | Account_Type | Balance |
+-----+-----+-----+-----+-----+
| 101 | Aishwarya | 10001 | Savings | 50000.00 |
| 102 | Priya | 10002 | Savings | 35000.00 |
| 103 | Rahul | 10003 | Current | 75000.00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)

mysql> CREATE VIEW Customer_Transaction_View AS
-> SELECT
-> c.Customer_ID,
-> c.Customer_Name,
-> a.Account_No,
-> t.Transaction_ID,
-> t.Transaction_Type,
-> t.Amount,
-> t.Transaction_Date
-> FROM Customer c
-> JOIN Account a
-> ON c.Customer_ID = a.Customer_ID
-> JOIN Bank_Transaction t
-> ON a.Account_No = t.Account_No;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Customer_Transaction_View;
Empty set (0.01 sec)

mysql> CREATE VIEW Manager_Account_View AS
-> SELECT
-> c.Customer_ID,
-> c.Customer_Name,
-> c.Email,
-> c.Phone,
-> a.Account_No,
-> a.Account_Type,
-> a.Balance
-> FROM Customer c
-> JOIN Account a
-> ON c.Customer_ID = a.Customer_ID;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Manager_Account_View;
+-----+-----+-----+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Email | Phone | Account_No | Account_Type | Balance |
+-----+-----+-----+-----+-----+-----+-----+
| 101 | Aishwarya | aishwarya@gmail.com | 9876543210 | 10001 | Savings | 50000.00 |
| 102 | Priya | priya@gmail.com | 9876543211 | 10002 | Savings | 35000.00 |
| 103 | Rahul | rahul@gmail.com | 9876543212 | 10003 | Current | 75000.00 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

```

## 5)Sales Database Normalization up to 3NF / BCNF

### 1. Aim

To analyze a sales database containing redundant data, identify **insertion, update, and deletion anomalies**, and normalize the database up to **Third Normal Form (3NF)** and verify the resulting relations with respect to **BCNF**.

### 2. Unnormalized Sales Relation

Suppose the company stores all sales information in one table:

#### SALES

| Order_ID | Order_Date | Customer_ID | Customer_Name | Customer_Phone | Product_ID | Product_Name | Unit_Price | Quantity | Salesperson_ID | Salesperson_Name |
|----------|------------|-------------|---------------|----------------|------------|--------------|------------|----------|----------------|------------------|
| O101     | 2026-08-01 | C01         | Aishwarya     | 9876543210     | P01        | Laptop       | 60000      | 1        | S01            | Ravi             |
| O101     | 2026-08-01 | C01         | Aishwarya     | 9876543210     | P02        | Mouse        | 1000       | 2        | S01            | Ravi             |
| O102     | 2026-08-02 | C02         | Priya         | 9876543211     | P01        | Laptop       | 60000      | 1        | S02            | Arun             |
| O103     | 2026-08-03 | C03         | Rahul         | 9876543212     | P03        | Keyboard     | 2000       | 1        | S01            | Ravi             |

### 3. Functional Dependencies

The important functional dependencies are:

Order\_ID → Order\_Date, Customer\_ID, Salesperson\_ID

Customer\_ID → Customer\_Name, Customer\_Phone

Product\_ID → Product\_Name, Unit\_Price

Salesperson\_ID → Salesperson\_Name

(Order\_ID, Product\_ID) → Quantity

The candidate key is:

(Order\_ID, Product\_ID)

because one order can contain multiple products.

### 4. Problems in the Original Table

#### 1. Update Anomaly

If the price of the Laptop changes, every row containing P01 must be updated.

If one row is missed, inconsistent prices occur.

#### 2. Insertion Anomaly

A new product cannot easily be inserted if no order has been placed for that product.

#### 3. Deletion Anomaly

If the only order containing a particular product is deleted, information about that product may also be lost.

#### 4. Data Redundancy

Customer, product, and salesperson information is repeated across multiple rows.

### 5) First Normal Form (1NF)

The table is in 1NF if every attribute contains a single atomic value and there are no repeating groups.

The relation can be represented as:

```
SALES(  
    Order_ID,  
    Order_Date,  
    Customer_ID,  
    Customer_Name,  
    Customer_Phone,  
    Product_ID,  
    Product_Name,  
    Unit_Price,  
    Quantity,  
    Salesperson_ID,  
    Salesperson_Name  
)
```

Candidate key:

```
(Order_ID, Product_ID)
```

## **6) Second Normal Form (2NF)**

To achieve 2NF, remove partial dependencies.

Create separate relations.

### **ORDER**

```
ORDER(  
    Order_ID,  
    Order_Date,  
    Customer_ID,  
    Salesperson_ID  
)
```

### **PRODUCT**

```
PRODUCT(  
    Product_ID,  
    Product_Name,  
    Unit_Price  
)
```

## **ORDER\_ITEM**

```
ORDER_ITEM(  
    Order_ID,  
    Product_ID,  
    Quantity  
)
```

But customer and salesperson information still has transitive dependencies.

### **7)Third Normal Form (3NF)**

Remove transitive dependencies.

## **CUSTOMER**

```
CUSTOMER(  
    Customer_ID PK,  
    Customer_Name,  
    Customer_Phone  
)
```

## **SALESPERSON**

```
SALESPERSON(  
    Salesperson_ID PK,  
    Salesperson_Name  
)
```

## PRODUCT

```
PRODUCT(  
    Product_ID PK,  
    Product_Name,  
    Unit_Price  
)
```

## ORDERS

```
ORDERS(  
    Order_ID PK,  
    Order_Date,  
    Customer_ID FK,  
    Salesperson_ID FK  
)
```

## ORDER\_ITEM

```
ORDER_ITEM(  
    Order_ID PK/FK,  
    Product_ID PK/FK,  
    Quantity  
)
```



```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 112
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SalesDB;
Query OK, 1 row affected (0.09 sec)

mysql> USE SalesDB;
Database changed
mysql> DROP DATABASE SalesDB;
Query OK, 0 rows affected (0.13 sec)

mysql> CREATE DATABASE SalesDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SalesDB;
Database changed
mysql> CREATE TABLE Customer (
->   Customer_ID VARCHAR(10) PRIMARY KEY,
->   Customer_Name VARCHAR(50) NOT NULL,
->   Customer_Phone VARCHAR(15)
-> );
Query OK, 0 rows affected (0.14 sec)

mysql> CREATE TABLE Salesperson (
->   Salesperson_ID VARCHAR(10) PRIMARY KEY,
->   Salesperson_Name VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Product (
->   Product_ID VARCHAR(10) PRIMARY KEY,
```

```
-> Product_Name VARCHAR(50) NOT NULL,  
-> Unit_Price DECIMAL(10,2) NOT NULL,  
->  
-> CHECK (Unit_Price > 0)  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql> CREATE TABLE Orders (  
-> Order_ID VARCHAR(10) PRIMARY KEY,  
-> Order_Date DATE NOT NULL,  
-> Customer_ID VARCHAR(10) NOT NULL,  
-> Salesperson_ID VARCHAR(10) NOT NULL,  
->  
-> FOREIGN KEY (Customer_ID)  
-> REFERENCES Customer(Customer_ID),  
->  
-> FOREIGN KEY (Salesperson_ID)  
-> REFERENCES Salesperson(Salesperson_ID)  
-> );  
Query OK, 0 rows affected (0.09 sec)  
  
mysql> CREATE TABLE Order_Item (  
-> Order_ID VARCHAR(10),  
-> Product_ID VARCHAR(10),  
-> Quantity INT NOT NULL,  
->  
-> PRIMARY KEY (Order_ID, Product_ID),  
->  
-> FOREIGN KEY (Order_ID)  
-> REFERENCES Orders(Order_ID),  
->  
-> FOREIGN KEY (Product_ID)  
-> REFERENCES Product(Product_ID),  
->  
-> CHECK (Quantity > 0)  
-> );  
Query OK, 0 rows affected (0.11 sec)  
  
mysql> INSERT INTO Customer  
-> VALUES  
-> ('C01', 'Aishwarya', '9876543210'),
```

```
-> ('C02', 'Priya', '9876543211'),
-> ('C03', 'Rahul', '9876543212');
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Salesperson
-> VALUES
-> ('S01', 'Ravi'),
-> ('S02', 'Arun');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Product
-> VALUES
-> ('P01', 'Laptop', 60000),
-> ('P02', 'Mouse', 1000),
-> ('P03', 'Keyboard', 2000);
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Orders
-> VALUES
-> ('O101', '2026-08-01', 'C01', 'S01'),
-> ('O102', '2026-08-02', 'C02', 'S02'),
-> ('O103', '2026-08-03', 'C03', 'S01');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Order_Item
-> VALUES
-> ('O101', 'P01', 1),
-> ('O101', 'P02', 2),
-> ('O102', 'P01', 1),
-> ('O103', 'P03', 1);
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT
-> o.Order_ID,
-> o.Order_Date,
-> c.Customer_Name,
```

```

MySQL 8.0 Command Line Cli  x  +  v

-> p.Unit_Price,
-> oi.Quantity,
-> s.Salesperson_Name
-> FROM Orders o
-> JOIN Customer c
-> ON o.Customer_ID = c.Customer_ID
-> JOIN Salesperson s
-> ON o.Salesperson_ID = s.Salesperson_ID
-> JOIN Order_Item oi
-> ON o.Order_ID = oi.Order_ID
-> JOIN Product p
-> ON oi.Product_ID = p.Product_ID;

+-----+-----+-----+-----+-----+-----+
| Order_ID | Order_Date | Customer_Name | Product_Name | Unit_Price | Quantity | Salesperson_Name |
+-----+-----+-----+-----+-----+-----+
| 0101     | 2026-08-01 | Aishwarya     | Laptop       | 60000.00   | 1        | Ravi              |
| 0101     | 2026-08-01 | Aishwarya     | Mouse        | 1000.00    | 2        | Ravi              |
| 0103     | 2026-08-03 | Rahul         | Keyboard     | 2000.00    | 1        | Ravi              |
| 0102     | 2026-08-02 | Priya         | Laptop       | 60000.00   | 1        | Arun              |
+-----+-----+-----+-----+-----+-----+

4 rows in set (0.05 sec)

mysql> SELECT
-> o.Order_ID,
-> p.Product_Name,
-> oi.Quantity,
-> p.Unit_Price,
-> (oi.Quantity * p.Unit_Price) AS Total_Amount
-> FROM Orders o
-> JOIN Order_Item oi
-> ON o.Order_ID = oi.Order_ID
-> JOIN Product p
-> ON oi.Product_ID = p.Product_ID;

+-----+-----+-----+-----+-----+
| Order_ID | Product_Name | Quantity | Unit_Price | Total_Amount |
+-----+-----+-----+-----+-----+
| 0101     | Laptop       | 1        | 60000.00   | 60000.00     |
| 0101     | Mouse        | 2        | 1000.00    | 2000.00      |
| 0102     | Laptop       | 1        | 60000.00   | 60000.00     |
| 0103     | Keyboard     | 1        | 2000.00    | 2000.00      |
+-----+-----+-----+-----+-----+

```

```

MySQL 8.0 Command Line Cli  x  +  v

+-----+-----+-----+-----+-----+-----+
4 rows in set (0.04 sec)

mysql> π Customer_Name (CUSTOMER)

Nothing found
Please try to run 'help contents' for a list of all accessible topics

mysql> CREATE VIEW Sales_Details AS
-> SELECT
-> o.Order_ID,
-> o.Order_Date,
-> c.Customer_Name,
-> p.Product_Name,
-> p.Unit_Price,
-> oi.Quantity,
-> (p.Unit_Price * oi.Quantity) AS Total_Amount,
-> s.Salesperson_Name
-> FROM Orders o
-> JOIN Customer c
-> ON o.Customer_ID = c.Customer_ID
-> JOIN Salesperson s
-> ON o.Salesperson_ID = s.Salesperson_ID
-> JOIN Order_Item oi
-> ON o.Order_ID = oi.Order_ID
-> JOIN Product p
-> ON oi.Product_ID = p.Product_ID;
Query OK, 0 rows affected (0.06 sec)

mysql> SELECT * FROM Sales_Details;

+-----+-----+-----+-----+-----+-----+-----+
| Order_ID | Order_Date | Customer_Name | Product_Name | Unit_Price | Quantity | Total_Amount | Salesperson_Name |
+-----+-----+-----+-----+-----+-----+-----+
| 0101     | 2026-08-01 | Aishwarya     | Laptop       | 60000.00   | 1        | 60000.00     | Ravi              |
| 0101     | 2026-08-01 | Aishwarya     | Mouse        | 1000.00    | 2        | 2000.00      | Ravi              |
| 0103     | 2026-08-03 | Rahul         | Keyboard     | 2000.00    | 1        | 2000.00      | Ravi              |
| 0102     | 2026-08-02 | Priya         | Laptop       | 60000.00   | 1        | 60000.00     | Arun              |
+-----+-----+-----+-----+-----+-----+-----+

4 rows in set (0.04 sec)

```

## 6)Customers and Orders: SQL Joins and Subqueries

### 1. Aim

To use the relational model, relational algebra, SQL joins, subqueries, grouping, and views to retrieve meaningful information from a customer and order database, especially to identify frequent customers.

### 2. Relational Model

We use two relations:

## **CUSTOMER**

```
CUSTOMER(  
    Customer_ID PK,  
    Customer_Name,  
    City,  
    Email  
)
```

## **ORDERS**

```
ORDERS(  
    Order_ID PK,  
    Customer_ID FK,  
    Order_Date,  
    Order_Amount  
)
```

## **Relationship:**

CUSTOMER 1 ————— M ORDERS

Enter password: \*\*\*\*\*

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 113

Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> CREATE DATABASE SalesDB;
```

ERROR 1007 (HY000): Can't create database 'salesdb'; database exists

```
mysql> USE SalesDB;
```

Database changed

```
mysql> DROP DATABASE SalesDB;
```

Query OK, 6 rows affected (0.33 sec)

```
mysql> CREATE DATABASE SalesDB;
```

Query OK, 1 row affected (0.01 sec)

```
mysql> USE SalesDB;
```

Database changed

```
mysql> CREATE TABLE Customer (
```

```
->     Customer_ID INT PRIMARY KEY,
```

```
->     Customer_Name VARCHAR(50) NOT NULL,
```

```
->     City VARCHAR(50),
```

```
->     Email VARCHAR(100) UNIQUE
```

```
-> );
```

Query OK, 0 rows affected (0.12 sec)

```
mysql> CREATE TABLE Orders (
```

```
->     Order_ID INT PRIMARY KEY,
```

```
->     Customer_ID INT NOT NULL,
```

```
->     Order_Date DATE NOT NULL,
```

```
->     Order_Amount DECIMAL(10,2) NOT NULL,
```

```
->
```

```
->     FOREIGN KEY (Customer_ID)
```

```
MySQL 8.0 Command Line Cli x + v
-> REFERENCES Customer(Customer_ID),
->
-> CHECK (Order_Amount > 0)
-> );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Customer
-> (Customer_ID, Customer_Name, City, Email)
-> VALUES
-> (101, 'Aishwarya', 'Chennai', 'aishwarya@gmail.com'),
-> (102, 'Priya', 'Bangalore', 'priya@gmail.com'),
-> (103, 'Rahul', 'Hyderabad', 'rahul@gmail.com'),
-> (104, 'Sneha', 'Chennai', 'sneha@gmail.com'),
-> (105, 'Arun', 'Mumbai', 'arun@gmail.com');
Query OK, 5 rows affected (0.02 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Orders
-> (Order_ID, Customer_ID, Order_Date, Order_Amount)
-> VALUES
-> (1001, 101, '2026-08-01', 5000),
-> (1002, 101, '2026-08-05', 3000),
-> (1003, 101, '2026-08-10', 4500),
-> (1004, 102, '2026-08-02', 2500),
-> (1005, 102, '2026-08-08', 3500),
-> (1006, 103, '2026-08-03', 7000),
-> (1007, 104, '2026-08-04', 2000),
-> (1008, 104, '2026-08-09', 2500),
-> (1009, 105, '2026-08-06', 1000);
Query OK, 9 rows affected (0.05 sec)
Records: 9 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Customer;
+-----+-----+-----+-----+
| Customer_ID | Customer_Name | City | Email |
+-----+-----+-----+-----+
| 101 | Aishwarya | Chennai | aishwarya@gmail.com |
| 102 | Priya | Bangalore | priya@gmail.com |
| 103 | Rahul | Hyderabad | rahul@gmail.com |
| 104 | Sneha | Chennai | sneha@gmail.com |
| 105 | Arun | Mumbai | arun@gmail.com |
+-----+-----+-----+-----+
```

```
mysql> SELECT * FROM Orders;
```

| Order_ID | Customer_ID | Order_Date | Order_Amount |
|----------|-------------|------------|--------------|
| 1001     | 101         | 2026-08-01 | 5000.00      |
| 1002     | 101         | 2026-08-05 | 3000.00      |
| 1003     | 101         | 2026-08-10 | 4500.00      |
| 1004     | 102         | 2026-08-02 | 2500.00      |
| 1005     | 102         | 2026-08-08 | 3500.00      |
| 1006     | 103         | 2026-08-03 | 7000.00      |
| 1007     | 104         | 2026-08-04 | 2000.00      |
| 1008     | 104         | 2026-08-09 | 2500.00      |
| 1009     | 105         | 2026-08-06 | 1000.00      |

```
9 rows in set (0.03 sec)
```

```
mysql> SELECT
```

```
->     c.Customer_ID,  
->     c.Customer_Name,  
->     o.Order_ID,  
->     o.Order_Date,  
->     o.Order_Amount  
-> FROM Customer c  
-> JOIN Orders o  
->     ON c.Customer_ID = o.Customer_ID;
```

| Customer_ID | Customer_Name | Order_ID | Order_Date | Order_Amount |
|-------------|---------------|----------|------------|--------------|
| 101         | Aishwarya     | 1001     | 2026-08-01 | 5000.00      |
| 101         | Aishwarya     | 1002     | 2026-08-05 | 3000.00      |
| 101         | Aishwarya     | 1003     | 2026-08-10 | 4500.00      |
| 102         | Priya         | 1004     | 2026-08-02 | 2500.00      |
| 102         | Priya         | 1005     | 2026-08-08 | 3500.00      |
| 103         | Rahul         | 1006     | 2026-08-03 | 7000.00      |
| 104         | Sneha         | 1007     | 2026-08-04 | 2000.00      |
| 104         | Sneha         | 1008     | 2026-08-09 | 2500.00      |
| 105         | Arun          | 1009     | 2026-08-06 | 1000.00      |

```
9 rows in set (0.05 sec)
```



```
mysql> SELECT
->     c.Customer_ID,
->     c.Customer_Name,
->     COUNT(o.Order_ID) AS Total_Orders
-> FROM Customer c
-> JOIN Orders o
->   ON c.Customer_ID = o.Customer_ID
-> GROUP BY
->     c.Customer_ID,
->     c.Customer_Name
-> HAVING COUNT(o.Order_ID) > 2;
```

```
+-----+-----+-----+
| Customer_ID | Customer_Name | Total_Orders |
+-----+-----+-----+
|          101 | Aishwarya      |             3 |
+-----+-----+-----+
1 row in set (0.02 sec)
```

```
mysql> SELECT
->     Customer_ID,
->     Customer_Name
-> FROM Customer
-> WHERE Customer_ID IN
-> (
->     SELECT Customer_ID
->     FROM Orders
->     GROUP BY Customer_ID
->     HAVING COUNT(Order_ID) > 2
-> );
```

```
+-----+-----+
| Customer_ID | Customer_Name |
+-----+-----+
|          101 | Aishwarya      |
+-----+-----+
1 row in set (0.01 sec)
```

```
mysql> SELECT
->     c.Customer_ID,
->     c.Customer_Name,
->     COUNT(o.Order_ID) AS Total_Orders
-> FROM Customer c
```

```
-> JOIN Orders o
->   ON c.Customer_ID = o.Customer_ID
-> GROUP BY
->   c.Customer_ID,
->   c.Customer_Name
-> HAVING COUNT(o.Order_ID) >= 2;
```

| Customer_ID | Customer_Name | Total_Orders |
|-------------|---------------|--------------|
| 101         | Aishwarya     | 3            |
| 102         | Priya         | 2            |
| 104         | Sneha         | 2            |

3 rows in set (0.00 sec)

```
mysql> SELECT
->   c.Customer_ID,
->   c.Customer_Name,
->   SUM(o.Order_Amount) AS Total_Purchase
-> FROM Customer c
-> JOIN Orders o
->   ON c.Customer_ID = o.Customer_ID
-> GROUP BY
->   c.Customer_ID,
->   c.Customer_Name
-> ORDER BY Total_Purchase DESC
-> LIMIT 1;
```

| Customer_ID | Customer_Name | Total_Purchase |
|-------------|---------------|----------------|
| 101         | Aishwarya     | 12500.00       |

1 row in set (0.04 sec)

```
mysql> SELECT
->   c.Customer_ID,
->   c.Customer_Name,
->   SUM(o.Order_Amount) AS Total_Purchase
-> FROM Customer c
-> JOIN Orders o
->   ON c.Customer_ID = o.Customer_ID
```

```

-> GROUP BY
->     c.Customer_ID,
->     c.Customer_Name
-> HAVING SUM(o.Order_Amount) >
-> (
->     SELECT AVG(Total_Purchase)
->     FROM
->     (
->         SELECT
->             Customer_ID,
->             SUM(Order_Amount) AS Total_Purchase
->         FROM Orders
->         GROUP BY Customer_ID
->     ) AS CustomerTotals
-> );

```

| Customer_ID | Customer_Name | Total_Purchase |
|-------------|---------------|----------------|
| 101         | Aishwarya     | 12500.00       |
| 103         | Rahul         | 7000.00        |

2 rows in set (0.01 sec)

```

mysql> SELECT
->     c.Customer_ID,
->     c.Customer_Name
-> FROM Customer c
-> LEFT JOIN Orders o
->     ON c.Customer_ID = o.Customer_ID
-> WHERE o.Order_ID IS NULL;

```

Empty set (0.04 sec)

```

mysql> CREATE VIEW Frequent_Customers AS
-> SELECT
->     c.Customer_ID,
->     c.Customer_Name,
->     COUNT(o.Order_ID) AS Total_Orders,
->     SUM(o.Order_Amount) AS Total_Purchase
-> FROM Customer c
-> JOIN Orders o
->     ON c.Customer_ID = o.Customer_ID

```

```

->     ON c.Customer_ID = o.Customer_ID
-> GROUP BY
->     c.Customer_ID,
->     c.Customer_Name
-> HAVING COUNT(o.Order_ID) > 2;

```

Query OK, 0 rows affected (0.07 sec)

```
mysql> SELECT * FROM Frequent_Customers;
```

| Customer_ID | Customer_Name | Total_Orders | Total_Purchase |
|-------------|---------------|--------------|----------------|
| 101         | Aishwarya     | 3            | 12500.00       |

1 row in set (0.04 sec)

## **7)Decomposition of a Relation into 4NF**

### **1. Aim**

To analyze a relation containing multi-valued dependencies (MVDs) and decompose the relation into Fourth Normal Form (4NF) to eliminate redundancy and avoid update, insertion, and deletion anomalies.

### **2. Given Relation**

Consider a university relation:

|                                                       |
|-------------------------------------------------------|
| STUDENT_INFO(Student_ID, Student_Name, Course, Hobby) |
|-------------------------------------------------------|

Suppose a student can have multiple courses and multiple hobbies, and the courses and hobbies are independent of each other.

### **Sample Data**

| Student_ID | Student_Name | Course | Hobby   |
|------------|--------------|--------|---------|
| S101       | Aishwarya    | DBMS   | Drawing |
| S101       | Aishwarya    | DBMS   | Music   |
| S101       | Aishwarya    | Python | Drawing |
| S101       | Aishwarya    | Python | Music   |
| S102       | Priya        | Java   | Dancing |
| S102       | Priya        | Java   | Reading |

For student S101:

- Courses = {DBMS, Python}
- Hobbies = {Drawing, Music}

Because courses and hobbies are independent, every course is combined with every hobby.

### **3. Multi-Valued Dependencies**

The relation has these multi-valued dependencies:

|                                        |
|----------------------------------------|
| Student_ID $\twoheadrightarrow$ Course |
|----------------------------------------|

And

|                                       |
|---------------------------------------|
| Student_ID $\twoheadrightarrow$ Hobby |
|---------------------------------------|

### **4. Why Does the Relation Violate 4NF?**

## 4NF Rule

A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial multivalued dependency:

|                          |
|--------------------------|
| $X \twoheadrightarrow Y$ |
|--------------------------|

 X must be a **superkey**.

In our relation:

|                                                       |
|-------------------------------------------------------|
| STUDENT_INFO(Student_ID, Student_Name, Course, Hobby) |
|-------------------------------------------------------|

Student\_ID is **not a superkey**, because:

|                   |
|-------------------|
| Student_ID = S101 |
|-------------------|

does not uniquely identify one tuple.

There are four tuples for S101.

Therefore:

|                                        |
|----------------------------------------|
| Student_ID $\twoheadrightarrow$ Course |
|----------------------------------------|

violates 4NF.

Also:

Student\_ID  $\twoheadrightarrow$  Hobby

violates 4NF.

**Therefore:**

|                             |
|-----------------------------|
| STUDENT INFO is NOT in 4NF. |
|-----------------------------|

## 5. 4NF Decomposition

We decompose the relation based on the independent multi-valued dependencies.

**Relation 1: STUDENT\_COURSE**

|                                    |
|------------------------------------|
| STUDENT_COURSE(Student_ID, Course) |
|------------------------------------|

Data:

| Student_ID | Course |
|------------|--------|
|------------|--------|

|      |        |
|------|--------|
| S101 | DBMS   |
| S101 | Python |
| S102 | Java   |

## Relation 2: STUDENT\_HOBBY

STUDENT\_HOBBY(Student\_ID, Hobby)

Data:

| Student_ID | Hobby   |
|------------|---------|
| S101       | Drawing |
| S101       | Music   |
| S102       | Dancing |
| S102       | Reading |

## 6. What About Student\_Name?

Student\_Name depends only on Student\_ID:

Student\_ID  $\rightarrow$  Student\_Name

Therefore, to represent the dependency correctly and avoid repeating the student's name, we can create a separate relation:

## STUDENT

STUDENT(Student\_ID, Student\_Name)

| Student_ID | Student_Name |
|------------|--------------|
| S101       | Aishwarya    |
| S102       | Priya        |

Thus, the clean final design is:

STUDENT(Student\_ID, Student\_Name)  
STUDENT\_COURSE(Student\_ID, Course)  
STUDENT\_HOBBY(Student\_ID, Hobby)

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 114
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE University4NF;
Query OK, 1 row affected (0.04 sec)

mysql> USE University4NF;
Database changed
mysql> CREATE TABLE Student (
  ->     Student_ID VARCHAR(10) PRIMARY KEY,
  ->     Student_Name VARCHAR(50) NOT NULL
  -> );
Query OK, 0 rows affected (0.11 sec)

mysql> CREATE TABLE Student_Course (
  ->     Student_ID VARCHAR(10),
  ->     Course VARCHAR(50),
  ->     PRIMARY KEY (Student_ID, Course),
  ->     FOREIGN KEY (Student_ID)
  ->         REFERENCES Student(Student_ID)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Student_Hobby (
  ->     Student_ID VARCHAR(10),
  ->     Hobby VARCHAR(50),
  ->     PRIMARY KEY (Student_ID, Hobby),
  -> );
```

```
-> FOREIGN KEY (Student_ID)
-> REFERENCES Student(Student_ID)
-> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO Student VALUES
-> ('S101', 'Aishwarya'),
-> ('S102', 'Priya');
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Student_Course VALUES
-> ('S101', 'DBMS'),
-> ('S101', 'Python'),
-> ('S102', 'Java');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Student_Hobby VALUES
-> ('S101', 'Drawing'),
-> ('S101', 'Music'),
-> ('S102', 'Dancing'),
-> ('S102', 'Reading');
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT
-> s.Student_ID,
-> s.Student_Name,
-> sc.Course
-> FROM Student s
-> JOIN Student_Course sc
-> ON s.Student_ID = sc.Student_ID;
```

```
+-----+-----+-----+
| Student_ID | Student_Name | Course |
+-----+-----+-----+
| S101       | Aishwarya    | DBMS   |
| S101       | Aishwarya    | Python |
| S102       | Priya        | Java   |
+-----+-----+-----+
```

```
3 rows in set (0.00 sec)
```



```
mysql> SELECT
->     s.Student_ID,
->     s.Student_Name,
->     sh.Hobby
-> FROM Student s
-> JOIN Student_Hobby sh
->     ON s.Student_ID = sh.Student_ID;
```

| Student_ID | Student_Name | Hobby   |
|------------|--------------|---------|
| S101       | Aishwarya    | Drawing |
| S101       | Aishwarya    | Music   |
| S102       | Priya        | Dancing |
| S102       | Priya        | Reading |

4 rows in set (0.00 sec)

## 8)Evaluation of Integrity Constraints

### 1. Aim

To evaluate a poorly designed database schema, identify the causes of inconsistent data, and apply appropriate integrity constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT to maintain data accuracy and consistency.

### 2. Given Improper Schema

Consider the following university database:

```
STUDENT(
    Student_ID,
    Student_Name,
    Email,
    Department_ID,
    Department_Name,
    Age
)
```

**Example inconsistent data:**

| Student_ID | Student_Name | Email           | Department_ID | Department_Name | Age |
|------------|--------------|-----------------|---------------|-----------------|-----|
| 101        | Aishwarya    | aish@gmail.com  | D01           | IT              | 20  |
| 101        | Aishwarya    | aish@gmail.com  | D01           | IT              | 20  |
| 102        | Priya        | NULL            | D01           | CSE             | 19  |
| 103        | Rahul        | rahul@gmail.com | D05           | IT              | -5  |
| 104        | Sneha        | rahul@gmail.com | D02           | HR              | 25  |

```

MySQL 8.0 Command Line Cli  ×  +  ▾
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 115
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE UniversityIntegrityDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE UniversityIntegrityDB;
Database changed
mysql> DROP DATABASE UniversityIntegrityDB;
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE DATABASE UniversityIntegrityDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE UniversityIntegrityDB;
Database changed
mysql> CREATE TABLE Department (
  ->     Department_ID VARCHAR(10) PRIMARY KEY,
  ->     Department_Name VARCHAR(50) NOT NULL UNIQUE
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Student (
  ->     Student_ID INT PRIMARY KEY,
  ->     Student_Name VARCHAR(50) NOT NULL,
  ->     Email VARCHAR(100) NOT NULL UNIQUE,
  ->     Age INT NOT NULL,
  ->     Department_ID VARCHAR(10) NOT NULL,
  ->
  ->     CONSTRAINT chk_student_age
  ->         CHECK (Age >= 16 AND Age <= 100),

```

```
MySQL 8.0 Command Line Cli  ×  +  ▾

->
->     CONSTRAINT fk_student_department
->     FOREIGN KEY (Department_ID)
->     REFERENCES Department(Department_ID)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Department
-> (Department_ID, Department_Name)
-> VALUES
-> ('D01', 'IT'),
-> ('D02', 'CSE'),
-> ('D03', 'ECE'),
-> ('D04', 'HR');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student
-> (Student_ID, Student_Name, Email, Age, Department_ID)
-> VALUES
-> (101, 'Aishwarya', 'aishwarya@gmail.com', 20, 'D01'),
-> (102, 'Priya', 'priya@gmail.com', 19, 'D02'),
-> (103, 'Rahul', 'rahul@gmail.com', 21, 'D01'),
-> (104, 'Sneha', 'sneha@gmail.com', 20, 'D03');
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student
-> VALUES
-> (101, 'Arun', 'arun@gmail.com', 22, 'D02');
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
mysql> INSERT INTO Student
-> VALUES
-> (105, 'Arun', 'aishwarya@gmail.com', 22, 'D02');
ERROR 1062 (23000): Duplicate entry 'aishwarya@gmail.com' for key 'student.Email'
mysql> INSERT INTO Student
-> VALUES
-> (106, 'Arun', 'arun@gmail.com', -5, 'D02');
ERROR 3819 (HY000): Check constraint 'chk_student_age' is violated.
mysql> INSERT INTO Student
-> VALUES
```

```
-> (108, NULL, 'test@gmail.com', 20, 'D01');  
ERROR 1048 (23000): Column 'Student_Name' cannot be null
```

```
mysql> CREATE TABLE Student2 (  
->     Student_ID INT PRIMARY KEY,  
->     Student_Name VARCHAR(50) NOT NULL,  
->     Email VARCHAR(100) UNIQUE,  
->     Age INT NOT NULL DEFAULT 18,  
->     Department_ID VARCHAR(10) NOT NULL,  
->     CHECK (Age >= 16 AND Age <= 100),  
->     FOREIGN KEY (Department_ID)  
->         REFERENCES Department(Department_ID)  
-> );
```

```
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> INSERT INTO Student2  
-> (Student_ID, Student_Name, Email, Department_ID)  
-> VALUES  
-> (109, 'Meena', 'meena@gmail.com', 'D01');  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT  
->     s.Student_ID,  
->     s.Student_Name,  
->     s.Email,  
->     d.Department_Name  
-> FROM Student s  
-> JOIN Department d  
->     ON s.Department_ID = d.Department_ID;
```

| Student_ID | Student_Name | Email               | Department_Name |
|------------|--------------|---------------------|-----------------|
| 101        | Aishwarya    | aishwarya@gmail.com | IT              |
| 102        | Priya        | priya@gmail.com     | CSE             |
| 103        | Rahul        | rahul@gmail.com     | IT              |
| 104        | Sneha        | sneha@gmail.com     | ECE             |

```
4 rows in set (0.01 sec)
```

```
mysql> CREATE VIEW Student_Department_View AS
-> SELECT
->     s.Student_ID,
->     s.Student_Name,
->     d.Department_Name
-> FROM Student s
-> JOIN Department d
->     ON s.Department_ID = d.Department_ID;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> SELECT * FROM Student_Department_View;
+-----+-----+-----+
| Student_ID | Student_Name | Department_Name |
+-----+-----+-----+
|          101 | Aishwarya    | IT              |
|          102 | Priya        | CSE             |
|          103 | Rahul        | IT              |
|          104 | Sneha        | ECE             |
+-----+-----+-----+
4 rows in set (0.04 sec)
```

## 9)5NF – Join Dependency and Decomposition

### 1. Aim

To identify a **join dependency** in a relational schema and decompose the relation into **Fifth Normal Form (5NF)** to remove redundancy and maintain data consistency.

### 2. Given Relation

Consider the following relation:

SUPPLY(Employee\_ID, Project\_ID, Supplier\_ID)

Assume the following business rule:

An employee can work on several projects, a project can use several suppliers, and an employee can work with several suppliers. These relationships are independent.

### Sample Data

| Employee_ID | Project_ID | Supplier_ID |
|-------------|------------|-------------|
| E1          | P1         | S1          |
| E1          | P1         | S2          |
| E1          | P2         | S1          |
| E1          | P2         | S2          |
| E2          | P1         | S1          |
| E2          | P1         | S2          |
| E2          | P2         | S1          |
| E2          | P2         | S2          |

### 3. Join Dependency

A **join dependency** exists when a relation can be reconstructed exactly by joining two or more projections of that relation.

For the given relation:

|                                              |
|----------------------------------------------|
| SUPPLY(Employee_ID, Project_ID, Supplier_ID) |
|----------------------------------------------|

the join dependency is:

|                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{aligned} \text{SUPPLY} = & \\ & \pi(\text{Employee\_ID}, \text{Project\_ID})(\text{SUPPLY}) \\ & \bowtie \\ & \pi(\text{Project\_ID}, \text{Supplier\_ID})(\text{SUPPLY}) \\ & \bowtie \\ & \pi(\text{Employee\_ID}, \text{Supplier\_ID})(\text{SUPPLY}) \end{aligned}$ |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

In simple words, the original SUPPLY relation can be reconstructed from three smaller relations.

#### 4. Why Is There Redundancy?

For example, suppose:

E1 works on P1 and P2  
P1 and P2 use S1 and S2  
E1 works with S1 and S2

The original table must store several combinations:

E1 – P1 – S1  
E1 – P1 – S2  
E1 – P2 – S1  
E1 – P2 – S2

The same employee-project and supplier information is repeated.

This can cause:

- **Insertion anomaly**
- **Deletion anomaly**
- **Update anomaly**
- **Data redundancy**

#### 5. 5NF Rule

A relation is in **Fifth Normal Form (5NF)** if every non-trivial join dependency is implied by the candidate keys of the relation.

5NF is also called:

**Project-Join Normal Form (PJ/NF).**

If a relation has a non-trivial join dependency that is not implied by its keys, it must be decomposed.

---

#### 6. Analysis of the Given Relation

Original relation:

SUPPLY(Employee\_ID, Project\_ID, Supplier\_ID)

The non-trivial join dependency is:

\*(Employee\_ID, Project\_ID,  
Project\_ID, Supplier\_ID,  
Employee\_ID, Supplier\_ID)

Therefore, the relation contains a join dependency and has redundancy.

So, we decompose it into three relations.

## 7. Decomposition into 5NF

### Relation 1: EMPLOYEE\_PROJECT

EMPLOYEE\_PROJECT(  
Employee\_ID,  
Project\_ID  
)

| Employee_ID | Project_ID |
|-------------|------------|
| E1          | P1         |
| E1          | P2         |
| E2          | P1         |
| E2          | P2         |

### Relation 2: PROJECT\_SUPPLIER

PROJECT\_SUPPLIER(  
Project\_ID,  
Supplier\_ID  
)



| Project_ID | Supplier_ID |
|------------|-------------|
| P1         | S1          |
| P1         | S2          |
| P2         | S1          |
| P2         | S2          |

**Relation 3: EMPLOYEE\_SUPPLIER**

```
EMPLOYEE_SUPPLIER(  
    Employee_ID,  
    Supplier_ID  
)
```

```
MySQL 8.0 Command Line Cl... × + ▾
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 117
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE Company5NF;
Query OK, 1 row affected (0.05 sec)

mysql> USE Company5NF;
Database changed
mysql> DROP DATABASE Company5NF;
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE DATABASE Company5NF;
Query OK, 1 row affected (0.01 sec)

mysql> USE Company5NF;
Database changed
mysql> CREATE TABLE Employee_Project (
  ->     Employee_ID VARCHAR(10),
  ->     Project_ID VARCHAR(10),
  ->
  ->     PRIMARY KEY (Employee_ID, Project_ID)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Project_Supplier (
  ->     Project_ID VARCHAR(10),
  ->     Supplier_ID VARCHAR(10),
  ->
  ->     PRIMARY KEY (Project_ID, Supplier_ID)
  -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> CREATE TABLE Employee_Supplier (  
->     Employee_ID VARCHAR(10),  
->     Supplier_ID VARCHAR(10),  
->  
->     PRIMARY KEY (Employee_ID, Supplier_ID)  
-> );
```

Query OK, 0 rows affected (0.09 sec)

```
mysql> INSERT INTO Employee_Project VALUES  
-> ('E1', 'P1'),  
-> ('E1', 'P2'),  
-> ('E2', 'P1'),  
-> ('E2', 'P2');
```

Query OK, 4 rows affected (0.05 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> INSERT INTO Project_Supplier VALUES  
-> ('P1', 'S1'),  
-> ('P1', 'S2'),  
-> ('P2', 'S1'),  
-> ('P2', 'S2');
```

Query OK, 4 rows affected (0.05 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> INSERT INTO Employee_Supplier VALUES  
-> ('E1', 'S1'),  
-> ('E1', 'S2'),  
-> ('E2', 'S1'),  
-> ('E2', 'S2');
```

Query OK, 4 rows affected (0.04 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT  
->     ep.Employee_ID,  
->     ep.Project_ID,  
->     es.Supplier_ID  
-> FROM Employee_Project ep  
-> JOIN Project_Supplier ps  
->     ON ep.Project_ID = ps.Project_ID  
-> JOIN Employee_Supplier es  
->     ON ep.Employee_ID = es.Employee_ID
```

```

-> JOIN Employee_Supplier es
->     ON ep.Employee_ID = es.Employee_ID
->     AND es.Supplier_ID = ps.Supplier_ID;
+-----+-----+-----+
| Employee_ID | Project_ID | Supplier_ID |
+-----+-----+-----+
| E1          | P1         | S1          |
| E1          | P1         | S2          |
| E1          | P2         | S1          |
| E1          | P2         | S2          |
| E2          | P1         | S1          |
| E2          | P1         | S2          |
| E2          | P2         | S1          |
| E2          | P2         | S2          |
+-----+-----+-----+
8 rows in set (0.00 sec)

```

## 10)SQL Query Optimization

### Aim

To analyze inefficient SQL queries and optimize them using **joins, indexing, and query restructuring**.

### 1. Inefficient Query

```

SELECT *
FROM Employee, Department
WHERE Employee.Department_ID = Department.Department_ID
AND Department.Department_Name = 'IT';

```

### Optimized Query Using JOIN

```

SELECT e.Employee_ID, e.Employee_Name, d.Department_Name
FROM Employee e
JOIN Department d
ON e.Department_ID = d.Department_ID
WHERE d.Department_Name = 'IT';

```

**Advantage:** Explicit JOIN improves readability and avoids unnecessary columns.

## 2. Indexing

Create indexes on frequently searched and joined columns:

```
CREATE INDEX idx_employee_dept
```

```
ON Employee(Department_ID);
```

```
CREATE INDEX idx_department_name
```

```
ON Department(Department_Name);
```

**Advantage:** Indexes reduce the number of rows that must be searched.

---

## 3. Avoid SELECT \*

✗ Inefficient:

---

## 4. Query Restructuring

✗ Inefficient:

```
SELECT Employee_Name  
FROM Employee  
WHERE Department_ID IN  
(  
    SELECT Department_ID  
    FROM Department  
    WHERE Department_Name = 'IT'  
);
```

✓ Optimized using JOIN:

```
SELECT e.Employee_Name  
FROM Employee e  
JOIN Department d  
ON e.Department_ID = d.Department_ID  
WHERE d.Department_Name = 'IT';
```